



Universidade do Minho
Escola de Engenharia

Inteligência Artificial

Trabalho Prático

Licenciatura em Engenharia Informática
Ano Letivo 2025/2026

Grupo 04

106919	Eduardo Freitas Fernandes
106842	Gonçalo Rodrigues Ribeiro
106888	José Mário Raimundo Lima
108961	Rodrigo Novais da Silva

Avaliação por Pares

A106919 Eduardo Freitas Fernandes DELTA = 0
A106842 Gonalo Rodrigues Ribeiro DELTA = 0
A106888 Jos   M  rio Raimundo Lima DELTA = 0
A108961 Rodrigo Novais da Silva DELTA = 0

Conte do

1	Introdu��o	2
2	Formula��o do Problema	2
2.1	Tipo de Problema	2
2.2	Representa��o de Estado	2
2.3	Estado Inicial	2
2.4	Estado Objetivo	2
2.5	Operadores	3
2.6	Fun��o de Custo	3
3	Tarefas Realizadas	3
4	Representa��o da Cidade	6
4.1	Carateriza��o da Frota	6
5	Estrat��gias de Procura	7
5.1	Procura N��o Informada	7
5.2	Procura Informada	7
6	Estat��sticas	8
7	Conclus��o	9

1 Introdução

Este projeto tem como objetivo o desenvolvimento de um sistema inteligente para a gestão e otimização de uma frota de táxis mista (veículos elétricos e a combustão) da empresa **TaxiGreen**.

O problema central reside na alocação eficiente de veículos a pedidos dinâmicos num ambiente urbano, considerando restrições de autonomia, tempos de recarga (no caso de veículos elétricos) e preferências dos clientes (custo, rapidez ou até preferência ambiental).

A solução desenvolvida modela um conjunto de zonas da cidade de Guimarães como um grafo e implementa algoritmos de procura para calcular rotas e tomar decisões de reabastecimento em tempo real.

2 Formulação do Problema

2.1 Tipo de Problema

O problema é formulado como um **problema determinístico, totalmente observável e de estado único**. Para cada pedido de transporte, o sistema conhece integralmente o estado atual do ambiente, nomeadamente a topologia da cidade, a localização dos veículos e as características relevantes para a tomada de decisão.

Nesse contexto, o **TaxiGreenGestor**, funciona como um gestor de frota, que coordena as ações dos veículos para satisfazer os pedidos

2.2 Representação de Estado

A cidade é representada sob a forma de um grafo, onde os nós representam localizações geográficas estratégicas e as arestas simbolizam as vias de circulação que as interligam.

Cada nó, instanciado através da classe **Localizacao**, caracteriza um ponto de interesse (como zonas de recolha de passageiros). Adicionalmente, estes nós podem ser tipificados como infraestruturas de suporte – **postos de recarga elétrica ou bombas de combustível** –, possuindo atributos específicos para a gestão da capacidade de atendimento, nomeadamente o número de lugares disponíveis e a ocupação temporal dos mesmos.

As arestas do grafo são ponderadas por duas variáveis distintas: a **distância espacial** (em metros) e o **custo temporal de travessia** (em segundos). Esta dualidade de pesos permite que os algoritmos de procura operem segundo diferentes critérios de **otimização**, em função da métrica de custo considerada.

O estado de cada veículo é definido pela sua localização atual, autonomia restante, capacidade de passageiros e estado de disponibilidade

2.3 Estado Inicial

O estado inicial corresponde à localização atual do veículo selecionado para satisfazer o pedido, ou, alternativamente, à localização de origem do pedido.

2.4 Estado Objetivo

O estado objetivo corresponde à localização de destino do pedido. Em cenários de gestão de autonomia, o objetivo pode também corresponder a um posto de abastecimento ou recarga compatível com o tipo de motorização do veículo.

2.5 Operadores

Os operadores possíveis são:

- `mover(origem, destino)`: Desloca o veículo entre nodos adjacentes, consumindo autonomia proporcional à distância e incrementando o tempo decorrido. A ação só é válida se $autonomia_{atual} > custo_{aresta}$.
- `abastecer(posto, tempo)`: Restaura a autonomia ao máximo, consumindo tempo de operação e ocupando um lugar no posto.
- `repor_pedido()`: Em caso de falha na alocação, o pedido é reintroduzido na fila com penalização temporal (atraso de 5 segundos).
- `rejeitar_pedido()`: Remove permanentemente o pedido da fila se o tempo de espera exceder o limite estipulado pelo cliente (horário), contabilizando-o como não atendido.

2.6 Função de Custo

A função de custo é definida de forma dinâmica, de acordo com a preferência associada a cada pedido de transporte. Quando a preferência do pedido é **LOW_COST**, o custo corresponde à distância total percorrida ao longo do percurso, medida em metros, multiplicada pelo custo operacional do veículo. Quando a preferência é **FAST**, o custo corresponde ao tempo total de viagem, medido em segundos, incorporando as variações de trânsito simuladas no sistema.

O custo de uma solução corresponde à soma dos custos associados a cada transição entre localizações ao longo do caminho encontrado. Desta forma, os algoritmos de procura selecionam a solução que minimiza a métrica de custo relevante para cada pedido.

3 Tarefas Realizadas

Para a concretização do sistema, foram implementados diversos componentes que permitem simular o funcionamento da frota e do ambiente urbano.

Gerador de Pedidos A simulação da procura de transporte é assegurada pela classe `GestorPedidos`, que atua como um gerador de carga dinâmico para o sistema. Este módulo é responsável por criar, armazenar e disponibilizar pedidos de transporte ao longo do tempo, seguindo uma lógica estocástica.

- **Geração Estocástica:** A criação de novos pedidos ocorre em intervalos aleatórios entre 1 e 3 segundos, simulando a chegada irregular de clientes. Cada pedido (`Pedido`) é instanciado com atributos aleatórios, garantindo diversidade nos testes:
 - **Origem/Destino:** Selecionados aleatoriamente da lista de locais da cidade.
 - **Passageiros:** Inteiro aleatório entre 1 e 8.
 - **Janela Temporal:** O cliente define um tempo máximo de espera entre 10 e 30 segundos.
 - **Motorização e Preferência:** Escolha aleatória entre Elétrico/Combustão e Low Cost/Fast.
- **Sistema de Filas e Prioridade:** O gestor implementa uma política de priorização baseada em duas filas distintas (*deques*): `pedidos_premium` e `pedidos_normais`. Existe uma probabilidade de 30% de um novo pedido ser classificado como **URGENTE** (inserido na fila premium) e 70% de ser **NORMAL**.

- **Escalonamento:** O método `proximo_pedido()` implementa a lógica de despacho. O sistema verifica sempre primeiro a existência de pedidos na fila *premium*. Apenas se esta estiver vazia é que são processados os pedidos da fila normal, garantindo assim a precedência dos clientes urgentes.
- **Mecanismo de Reintrodução:** Caso um pedido não possa ser satisfeito imediatamente (por falta de veículos disponíveis ou autonomia), o método `repor_pedido()` reintroduz o pedido na cabeça da respetiva fila (LIFO para re tentativas), permitindo que este seja processado novamente com prioridade máxima na próxima iteração.

Gerador de Trânsito A simulação do dinamismo urbano é implementada na classe `Graph`, permitindo que o ambiente evolua independentemente das ações dos agentes. Este módulo altera os pesos temporais das arestas do grafo em tempo real, simulando a fluidez ou congestionamento das vias de comunicação.

- **Perturbação Dinâmica:** A perturbação do trânsito ocorre de forma probabilística durante a execução dos algoritmos de procura. Sempre que o custo de uma aresta é consultado (método `get_edge_cost`), existe uma probabilidade de 1% de ser acionado um evento de mudança de trânsito, garantindo que o mapa não é estático.
- **Seleção Aleatória de Vias:** O algoritmo seleciona aleatoriamente uma localização (nó) da lista de locais disponíveis e, de seguida, escolhe aleatoriamente uma das suas conexões adjacentes (aresta). Isto assegura que qualquer via da cidade, seja central ou periférica, está sujeita a variações de tráfego.
- **Variação de Intensidade:** O sistema gera um valor de perturbação aleatório $\delta \in [1, 5]$ segundos. Existe uma probabilidade de 50% de este valor ser somado ao peso atual da aresta (simulando congestionamento) e 50% de ser subtraído (simulando alívio de tráfego).
- **Consistência Física:** Para evitar inconsistências no grafo, o método inclui salvaguardas na operação de subtração. Caso a redução do trânsito resultasse num tempo de travessia nulo ou negativo, o sistema ajusta o cálculo para garantir que o custo temporal da aresta permanece sempre positivo ($t \geq 1$), mantendo a validade matemática para os algoritmos de procura.

Gestor de Frota A coordenação central da operação é assegurada pela classe `TaxiGreenGestor`, que atua como o agente de decisão do sistema. Este componente processa sequencialmente as filas de pedidos e orquestra a alocação de recursos com base no estado global da rede e da frota.

- **Alocação Otimizada:** Para cada pedido pendente, o algoritmo avalia todos os veículos disponíveis com a motorização solicitada. O sistema calcula o custo de aproximação (do táxi até ao cliente) e seleciona o veículo que minimiza a função de custo total (custo de aproximação + custo da viagem). Esta decisão adapta-se dinamicamente à preferência do utilizador, otimizando por distância (LOW_COST) ou por tempo (FAST).
- **Gestão de Autonomia:** A preservação energética é validada antes de qualquer atribuição. Se um veículo não possuir autonomia suficiente para completar o serviço ou se o nível de energia for crítico ($< 10\%$), o gestor aciona o método `abastecer_veiculo`. Este procedimento identifica o posto de abastecimento compatível mais próximo e gera uma rota de desvio, bloqueando o veículo durante o tempo de recarga simulado.
- **Ciclo de Vida do Pedido:** O método `start` controla os estados dos pedidos. O gestor verifica timeouts (marcando pedidos expirados como REJEITADO) e gere a reintrodução de pedidos não atendidos na fila de espera (após aplicar uma penalização temporal), garantindo que falhas momentâneas de disponibilidade não resultam na perda imediata do cliente.

- **Monitorização Contínua:** Paralelamente ao despacho, o sistema executa rotinas de registo estatístico (método `registar_taxa_ocupacao`), amostrando periodicamente a percentagem de veículos ativos para permitir a análise de eficiência da frota pós-simulação.

Análise Estatística A avaliação de desempenho do sistema baseia-se num conjunto de métricas quantitativas, recolhidas e persistidas em ficheiro JSON após cada simulação. O módulo `estatisticas.py` processa estes dados para gerar visualizações comparativas entre os diferentes algoritmos de procura. As métricas analisadas permitem as seguintes comparações:

- **Eficiência Temporal:** O *Tempo Médio de Resposta* mede a latência entre a criação do pedido e a chegada ao destino. Esta métrica é fundamental para comparar a rapidez dos algoritmos (ex: A* vs BFS) na satisfação do cliente, especialmente sob condições de trânsito variável.
- **Gestão de Recursos:** A *Taxa de Ocupação Média* e a *Média de Lugares Vazios* indicam a eficiência da alocação. Uma taxa de ocupação elevada sugere um bom aproveitamento da frota, enquanto um número elevado de lugares vazios pode indicar que se estão a utilizar veículos grandes (ex: carrinhas de 9 lugares) para pedidos individuais, sinalizando ineficiência.
- **Sustentabilidade e Custos:** O *Custo Operacional Total* e as *Emissões de CO2* permitem avaliar o impacto económico e ambiental. Estas métricas são cruciais para verificar se a preferência por veículos elétricos (no algoritmo Greedy ou A* com heurística ajustada) resulta efetivamente numa redução da pegada ecológica em comparação com uma gestão cega.
- **Robustez do Serviço:** O *Número de Pedidos Rejeitados* e *Carros Bloqueados* medem a resiliência do sistema. Um valor alto de rejeições indica que o algoritmo é demasiado lento ou que a gestão da autonomia é ineficaz (carros parados a abastecer em momentos críticos), permitindo identificar gargalos na operação.
- **Otimização de Rotas:** A métrica *Metros sem Passageiros* quantifica a distância percorrida "em vazio" para ir buscar o cliente. Comparar este valor entre algoritmos revela qual a estratégia que melhor posiciona ou seleciona os veículos geograficamente, minimizando o desperdício de quilómetros.

4 Representação da Cidade

A topologia da cidade foi modelada através de um grafo ponderado, construído a partir de um conjunto de dados que representa localizações reais da cidade de Guimarães.

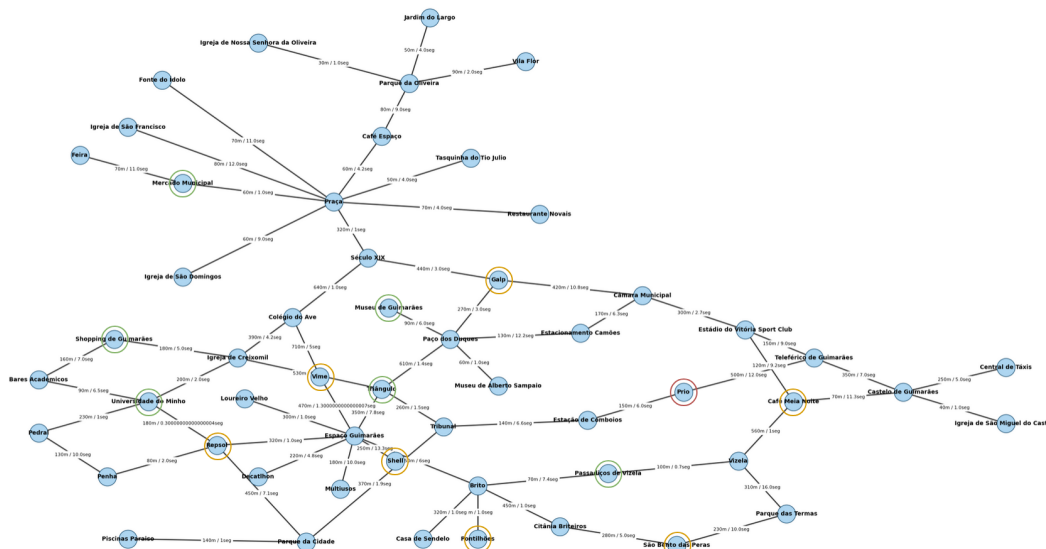


Figura 1: Representação da cidade de Guimarães.

Legenda: Círculos a verde — postos de recarga elétrica; Círculos a laranja — bombas de combustível; Círculos a vermelho — localizações com ambas as funcionalidades.

O grafo é composto por mais de 50 nodos (localizações) e as respetivas arestas (vias de circulação). Embora **todas** as localizações funcionem como pontos de recolha e entrega de passageiros, um subconjunto destes nodos funciona como postos de abastecimento:

- **Postos de Recarga Elétrica:** Locais como "Universidade do Minho", "Shopping de Guimarães" e "Prio", equipados com um número limitado de lugares de carregamento.
- **Bombas de Combustível:** Locais como "Galp", "Repsol" e "Shell", destinados ao reabastecimento dos veículos convencionais.

As arestas que conectam estes nós possuem pesos duplos – distância (metros) e tempo (segundos) – permitindo o cálculo de custos operacionais e temporais distintos.

4.1 Caraterização da Frota

A frota da *TaxiGreen* é constituída por 20 veículos heterogéneos, divididos entre motorização elétrica e a combustão. Cada veículo possui atributos específicos que influenciam a tomada de decisão do agente gestor:

- **Veículos a Combustão:** Modelos como *Toyota Corolla* ou *Dacia Sandero*. Caracterizam-se por uma elevada autonomia (aprox. 700-800 km) e tempos de reabastecimento reduzidos (2 a 10 segundos simulados). No entanto, apresentam emissões de CO2 positivas e custos operacionais variáveis.

- **Veículos Elétricos:** Modelos como *Tesla Model 3* ou *BMW Serie 3*. Distinguem-se pela ausência de emissões diretas ($emissao_CO2 = 0$). Em contrapartida, possuem autonomias inferiores (aprox. 300-500 km) e exigem tempos de imobilização superiores para recarga (15 a 25 segundos simulados).

Esta diversidade obriga o sistema a gerir compromissos entre a rapidez de serviço (favorecida pelos veículos a combustão em longas distâncias) e a sustentabilidade (favorecida pelos elétricos).

5 Estratégias de Procura

A navegação dos veículos no grafo da cidade é suportada por cinco algoritmos de procura distintos, implementados na classe `Graph`. A escolha do algoritmo e da função de custo é determinada dinamicamente com base nas preferências do pedido (rapidez vs economia).

5.1 Procura Não Informada

Nesta categoria, foram implementados três algoritmos que exploram o espaço de estados sem estimativas sobre a distância ao objetivo.

- **Breadth-First Search (BFS) e Depth-First Search (DFS):** Ambas as estratégias exploram o grafo com base na conectividade topológica, ignorando os pesos das arestas. A implementação do BFS utiliza uma estrutura de fila (`Queue`) para garantir a descoberta do caminho com menor número de nós, enquanto o DFS utiliza uma pilha (`stack`).
- **Uniform Cost Search (UCS):** Este algoritmo expande o nó com o menor custo de caminho acumulado $g(n)$, utilizando uma fila de prioridade (`heapq`). A principal particularidade desta implementação reside na definição dinâmica da função de custo $g(n)$, que varia consoante a preferência do cliente:
 - Se `LOW_COST`: $g(n)$ corresponde à distância acumulada (em metros).
 - Se `FAST`: $g(n)$ corresponde ao tempo de viagem acumulado (em segundos), incorporando as penalizações de trânsito geradas dinamicamente.

5.2 Procura Informada

Para orientar a procura na direção do objetivo e melhorar a eficiência, foram implementados algoritmos que utilizam uma função heurística $h(n)$.

Função Heurística Para ambos os algoritmos informados, definiu-se como heurística admissível a **Distância Euclidiana** entre as coordenadas do nó atual (x_n, y_n) e as coordenadas do nó destino (x_d, y_d) , calculada pelo método `get_heuristic`:

$$h(n) = \sqrt{(x_d - x_n)^2 + (y_d - y_n)^2}$$

Esta métrica estima a distância em linha reta, garantindo que nunca sobrestima o custo real em termos de distância.

- **Greedy Search:** Seleciona o próximo nó a expandir baseando-se exclusivamente na minimização de $h(n)$. A implementação mantém uma `open_list` e seleciona iterativamente o vizinho que apresenta a menor distância euclidiana ao destino, ignorando o custo do trajeto já percorrido.

- **A* (A-Star):** Combina o custo real do caminho com a estimativa heurística, minimizando a função de avaliação $f(n) = g(n) + h(n)$. Tal como no UCS, o componente $g(n)$ adapta-se à preferência do pedido (tempo ou distância). O algoritmo gere duas listas (`open_list` e `closed_list`) para assegurar que cada nó é avaliado e expandido de forma ótima, garantindo a melhor solução possível para a métrica de custo selecionada.

6 Estatísticas

O agente `TaxiGreenGestor` calcula as seguintes estatísticas (exemplo apresentado na figura 2):

- **Tempo Médio de Resposta:** este cálculo permite comparar os algoritmos perante a velocidade de resposta ao pedidos. Podemos verificar pela figura 2 que os algoritmos de procura não informados têm uma melhor performance relativamente aos algoritmos de procura informada, devendo-se ao facto de estes algoritmos não se guiarem pelos custos das ligações (exceto o de custo uniforme). O algoritmo de procura em profundidade tem melhores resultados como é verificável.
- **Taxa de ocupação média:** esta estatística permite analisar a percentagem de veículos ocupados, sendo registada a cada 10 segundos e calculada a média no fim. Todos os algoritmos aparentam ter uma performance uniforme nesta estatística.
- **Custo Operacional:** este gráfico indica o custo operacional total de cada algoritmo, e podemos verificar que o algoritmo de procura em profundidade tem resultados piores, dado que é mais provável que o caminho seja mais profundo, comparado a outros algoritmos.
- **Emissão de CO2:** esta estatística apresenta as emissões de CO2 totais, e mais uma vez a procura em profundidade falha em comparação aos outros algoritmos.
- **Metros sem passageiros:** este cálculo permite comparar os algoritmos sobre a distância percorrida sem passageiros. O algoritmo de procura em profundidade, mais uma vez, tem uma performance má, pois tem a tendência a calcular caminhos mais longos, percorrendo maiores distâncias.
- **Média de lugares vazios:** esta estatística apresenta a percentagem média de lugares vazios nos veículos. Podemos verificar que todos os algoritmos têm valores semelhantes, dado que não foi criado nenhuma gestão relativa aos lugares ocupados.
- **Número de pedidos rejeitados:** esta estatística apresenta o número de pedidos rejeitados, e podemos verificar que num total de 100 pedidos, nenhum foi rejeitado, ou seja, todos os algoritmos conseguiram resolver todos os pedidos.
- **Número de carros bloqueados:** este gráfico apresenta o número de veículos que ficaram bloqueados, isto é, chegaram a um nível de autonomia que não os permite alcançar nenhum posto de recarga ou bomba de combustível. Nenhum veículo ficou bloqueado, logo temos uma boa gestão por parte do agente.

É importante realçar que os pedidos realizados pelos algoritmos são aleatórios e diferentes, isto é, um algoritmo nunca irá resolver os mesmos pedidos que outro algoritmo, logo não é possível comparar os algoritmos em detalhe, mas de forma global podemos observar certas diferenças. Um exemplo disso é o algoritmo de procura em profundidade, que embora seja rápido a responder pedidos, as rotas calculadas são longas, obtendo custos elevados.

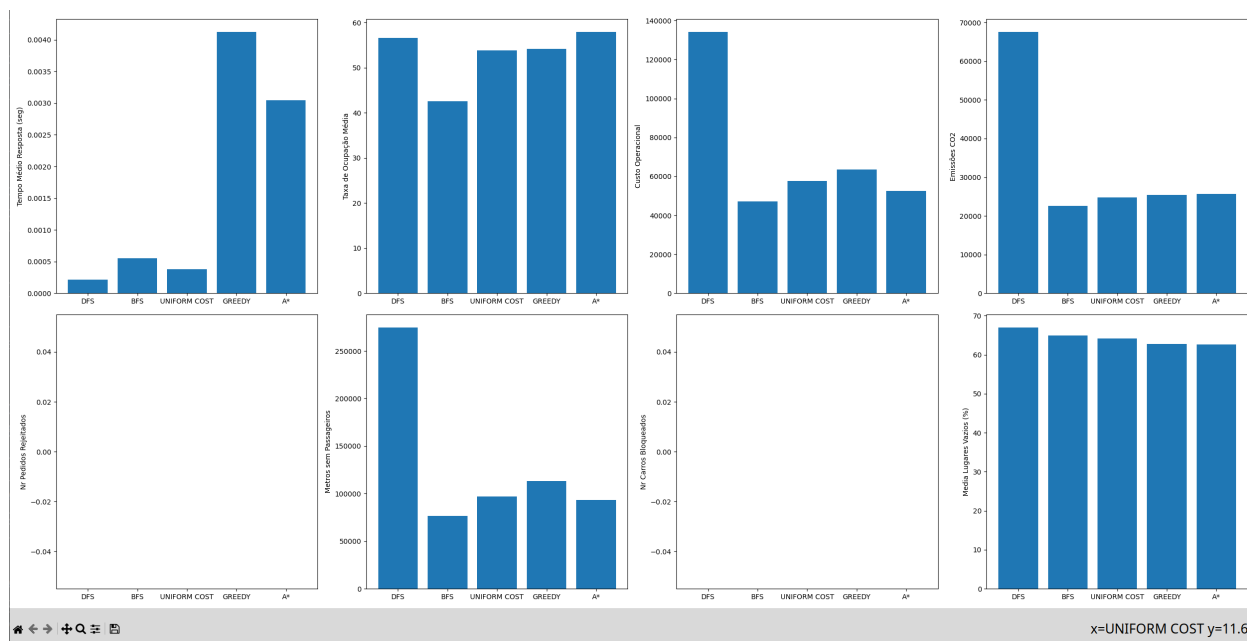


Figura 2: Estatísticas

7 Conclusão

Neste trabalho prático foi desenvolvido um sistema inteligente para a gestão e otimização de uma frota de táxis mista (elétricos e a combustão) na cidade de Guimarães. A implementação permitiu aplicar conceitos fundamentais de Inteligência Artificial, nomeadamente a aplicação de algoritmos de procura em grafos, a utilização de heurísticas e a modelação de ambientes dinâmicos.

A estrutura do sistema, controlada pelo *TaxiGreenGestor*, permitiu coordenar múltiplos veículos e pedidos concorrentes de forma eficiente. A modelação da cidade como um grafo ponderado, distinguindo custos temporais e espaciais, garantiu flexibilidade ao sistema, possibilitando a otimização de rotas segundo critérios distintos (rapidez ou economia) num mesmo ambiente operacional.

Decisões como a adoção da distância euclidiana como heurística admissível e a gestão proativa da autonomia provaram ser essenciais para viabilizar a operação da frota elétrica sem comprometer a disponibilidade do serviço. A estratégia de reabastecimento dinâmico permitiu mitigar as limitações de alcance dos veículos elétricos.

Apesar da eficácia demonstrada, o sistema atual realiza uma alocação exclusiva de um veículo por pedido. Como melhoria futura, sugere-se a implementação de um mecanismo de *ride-sharing*, permitindo que o mesmo veículo transporte passageiros de pedidos distintos com rotas compatíveis, o que aumentaria significativamente a taxa de ocupação e a eficiência global da frota.